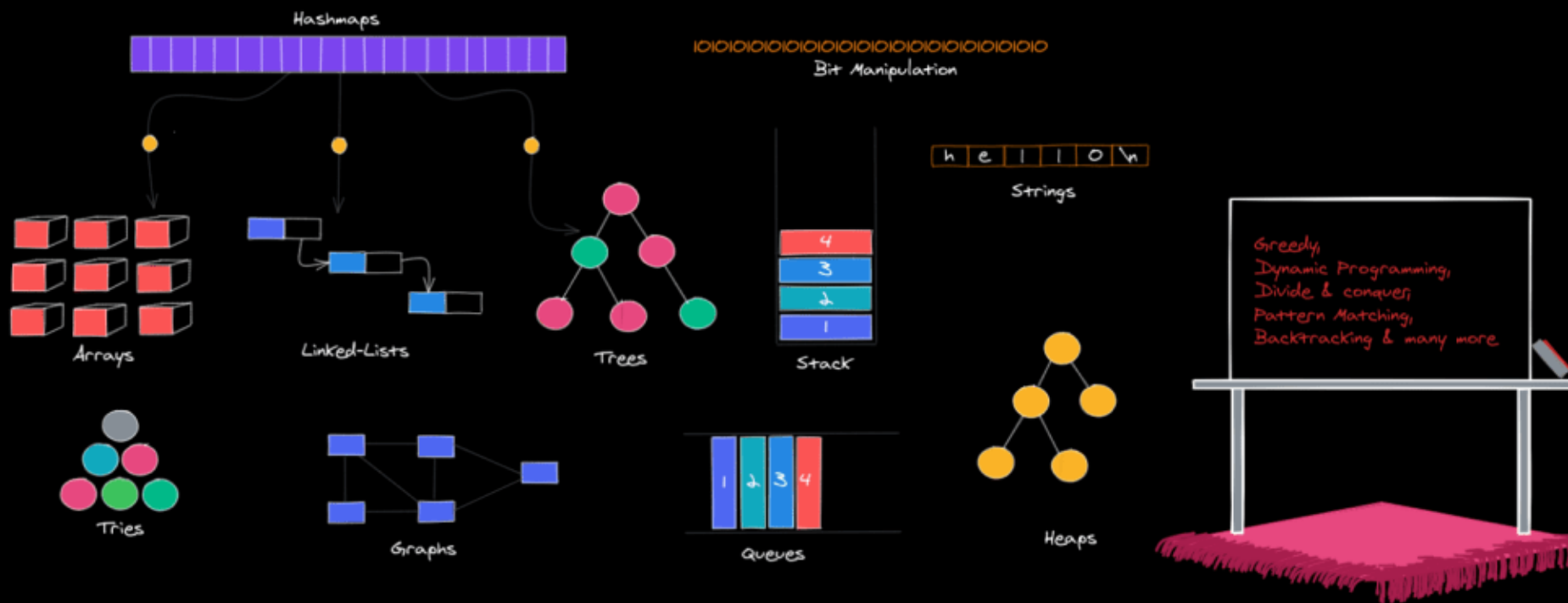




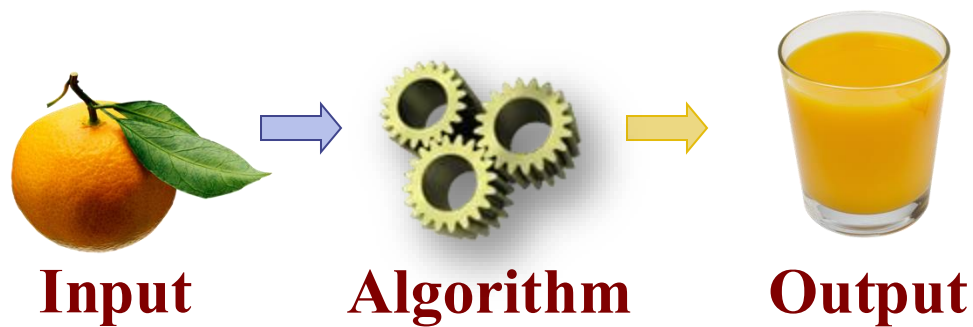
数据结构与算法导论

刘子博

FDSM

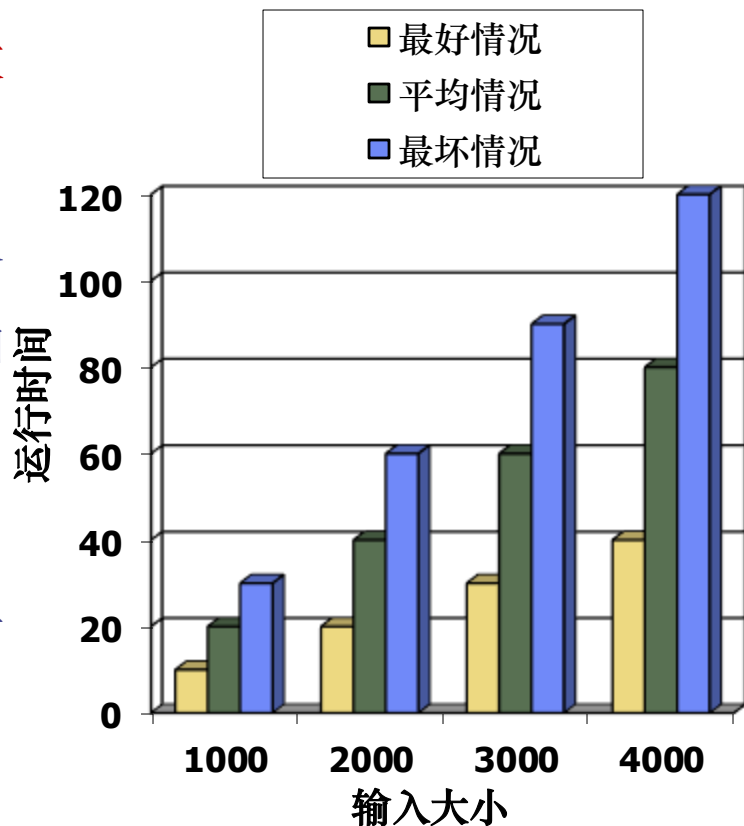
liuzibo@fudan.edu.cn

算法分析



程序运行时间

- 算法的**运行时间**至关重要，这是算法**效率**的体现
- 大部分算法将输入对象转化为输出对象
- 算法运行的时间通常随**输入大小**的增加而增加
- 算法运行的平均时间计算通常较困难
- 很多时候，我们关注**最坏情况**下的运行时间
 - 易于分析
 - 在金融、数据分析、游戏等领域很重要

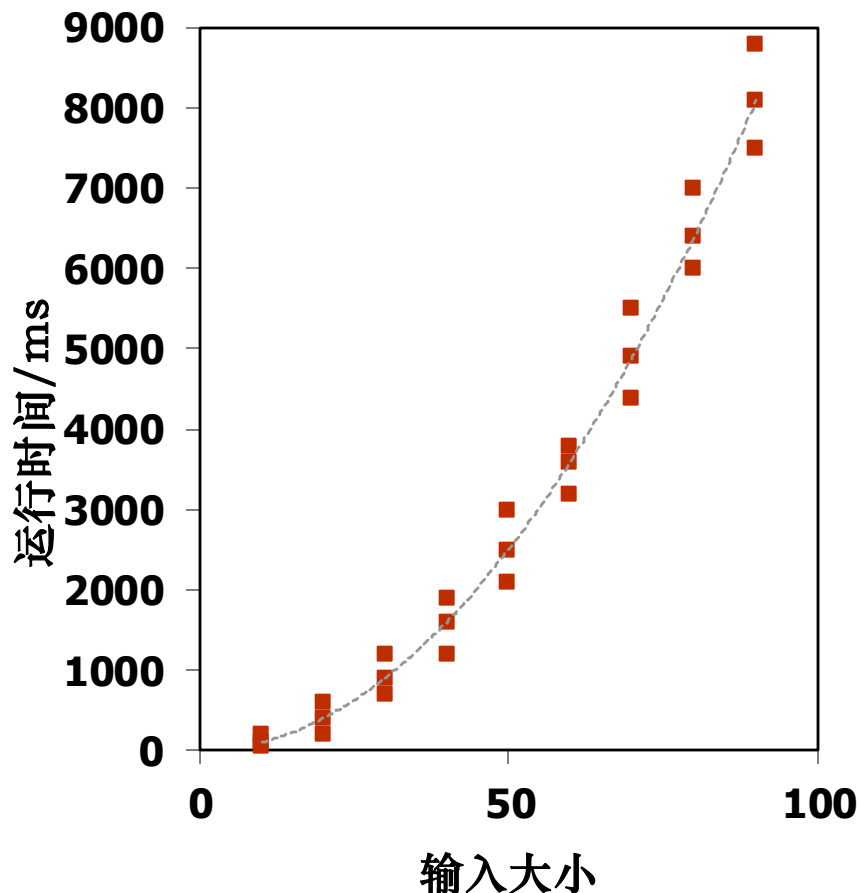


实验研究

- 我们可以通过**实验**记录算法运行时间与输入规模间的规律：
 - 写一段运行该算法的程序
 - 以不同输入（改变大小或输入的内容）运行程序，记录程序的运行时间，例：

```
from time import time
start_time = time( )
run algorithm
end_time = time( )
elapsed = end_time - start_time
```

- 绘制算法运行的性能图



实验的局限性

- 算法必须**完全实现**出来，而这可能**很困难**
- 实验结果仅由实验输入决定，这导致实验结果对算法在其他输入上的表现**不具有代表性**
- 为了比较两个算法，它们必须在**同样的软件+硬件**环境下执行



理论分析



- 我们可以使用理论分析的方式对算法效率进行分析，其优点为：
 - 使用算法的高层次描述，而不需要算法的实现
 - 将程序的运行时间表示为输入大小 n 的函数
 - 考虑所有可能的输入情况
 - 使得我们不必在特定的软件+硬件环境下评估算法的运行速度

伪代码

- 我们将使用伪代码的方式来描述算法，其特点为：
 - 伪代码为对算法的**高层次描述** (High-level description)
 - 比自然语言更加具有**结构性**
 - 比程序代码的**细节少**
 - 倾向于使用**符号**描述算法
 - 隐藏程序设计上的问题

伪代码细节示例



- 控制流
 - **if ... then ... [else ...]**
 - **while ... do ...**
 - **repeat ... until ...**
 - **for ... do ...**
 - 使用缩进来控制结构体
- 方法声明

Algorithm *method* (*arg* [, *arg*...])

Input ...

Output ...

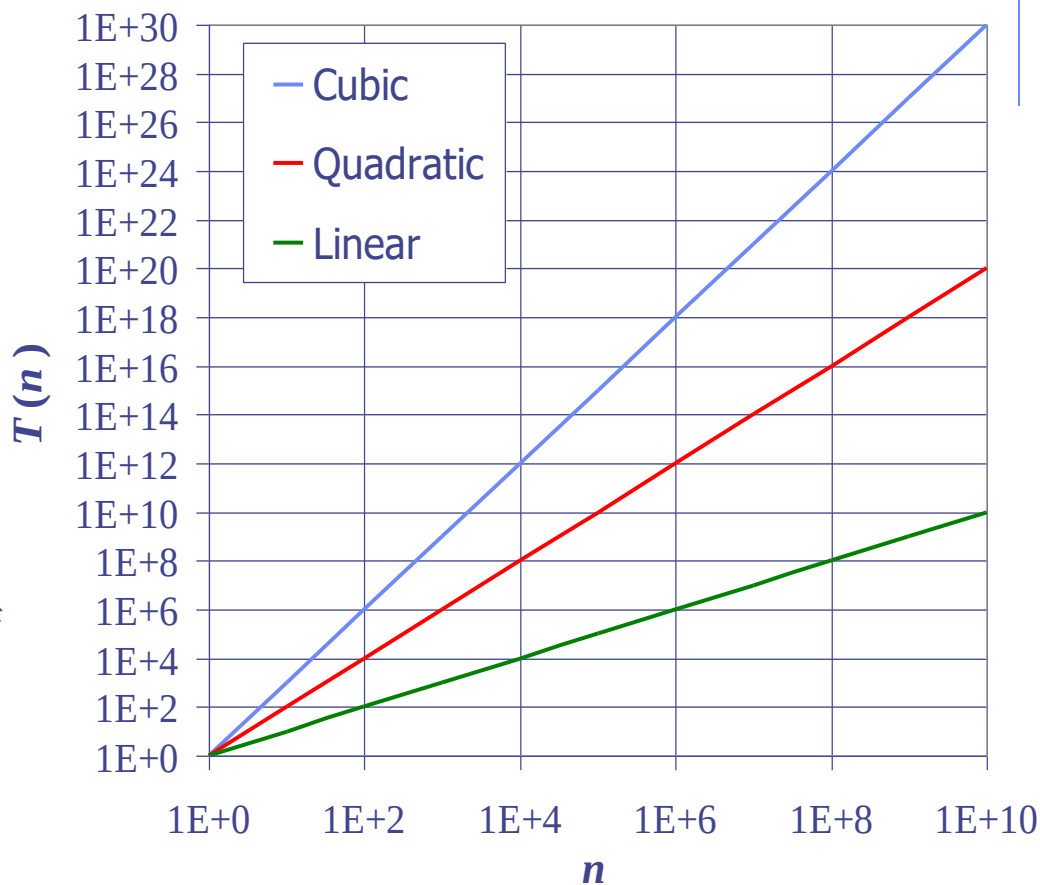
- 方法调用
method (*arg* [, *arg*...])
- 返回值
return *expression*
- 一些表达式与运算符:
 - ← 赋值
 - = 检验是否相等
 - n^2 上下标等数学表达格式
均可出现

7种重要函数

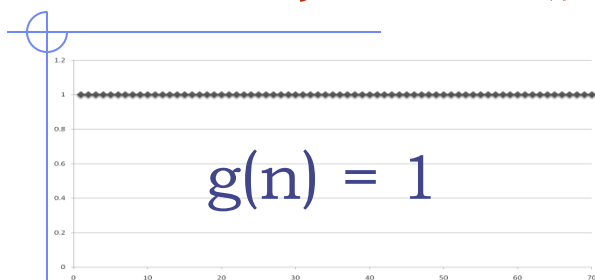
• 算法分析中常见的7种函数为：

- 常数函数 ≈ 1
- 对数函数 $\approx \log n$
- 线性函数 $\approx n$
- n -log- n 函数 $\approx n \log n$
- 二次函数 $\approx n^2$
- 三次函数 $\approx n^3$
- 指数函数 $\approx 2^n$

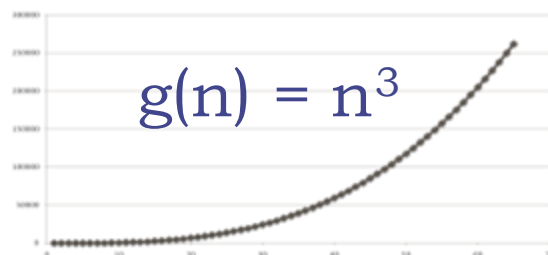
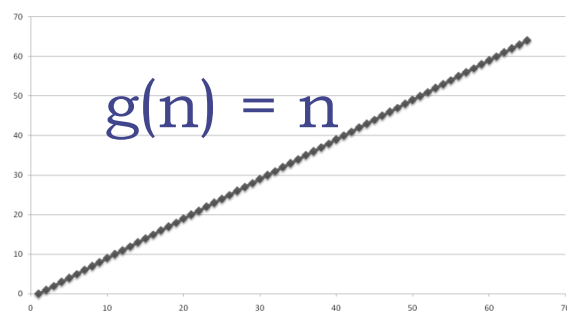
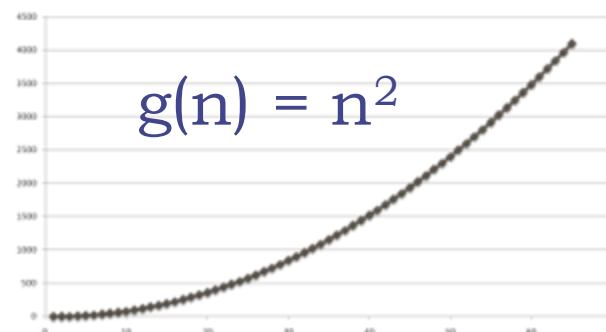
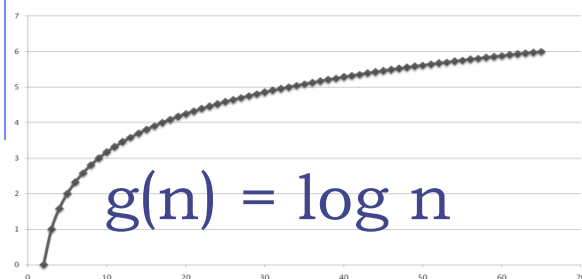
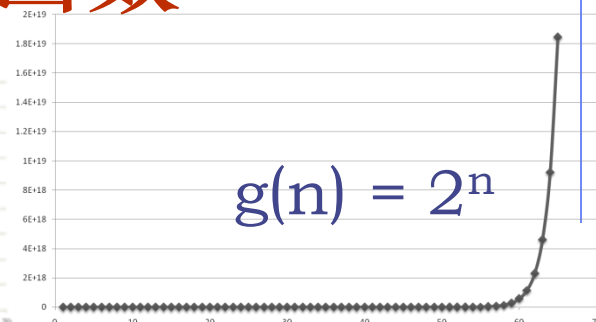
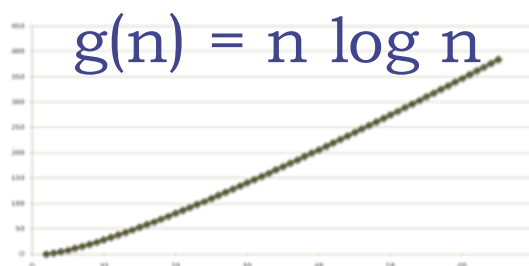
• 在双对数图表中，线的斜率与多项式函数的幂指数相对应



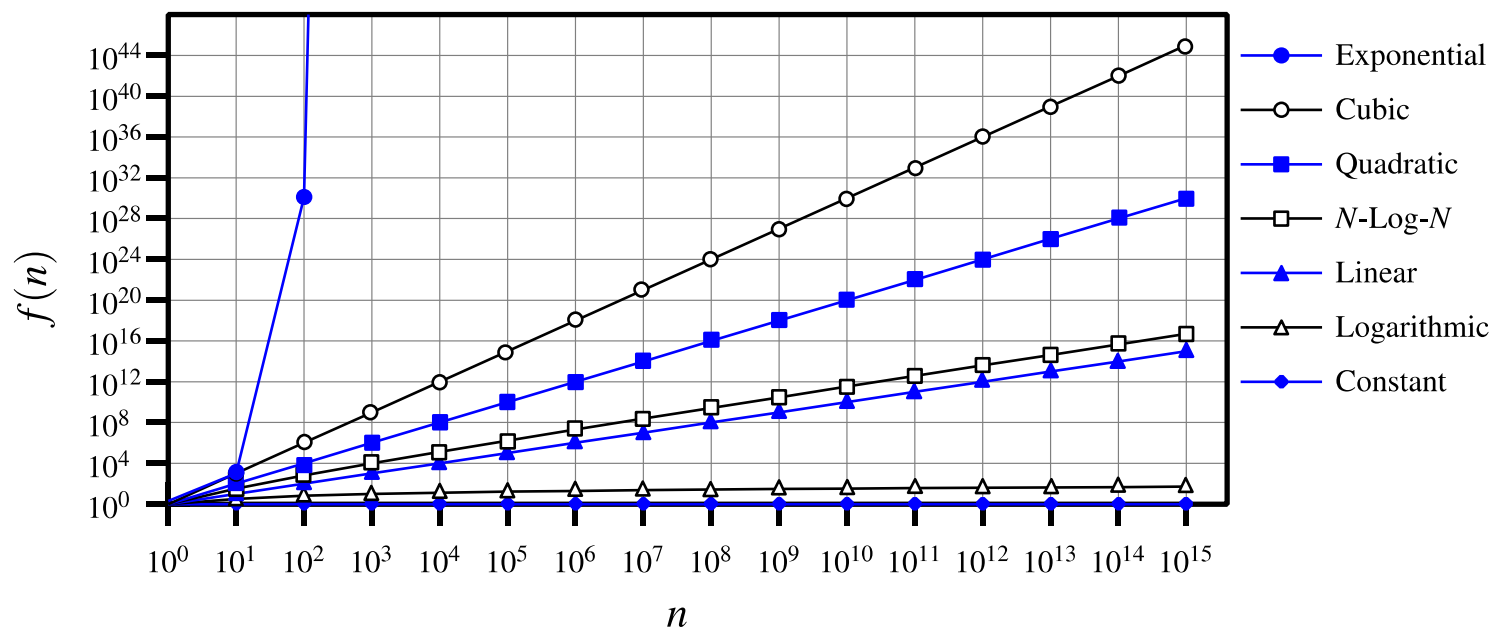
“正常” 刻度坐标下的7种函数



$$g(n) = n \log n$$



双对数刻度坐标下的7种函数



原子操作

- 原子操作是算法进行的一些基本运算
- 在伪代码中可被识别出来
- 很大程度上独立于编程语言而存在
- 原子操作的精确定义并不重要（我们马上会看到这是为什么）
- 执行时间为常数



- 例：
 - 计算一个表达式
 - 对一个变量赋值
 - 通过索引访问列表的一个元素
 - 调用一个函数（不包括内部执行）
 - 从函数返回一个值

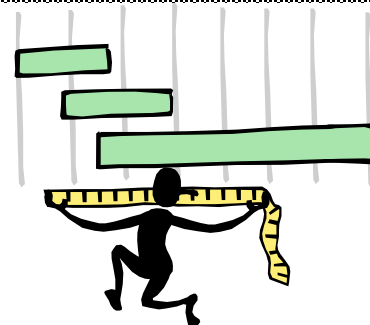
计算原子操作的次数

- 通过检查伪代码，我们可以确定算法执行的原子操作的最大次数，此最大次数为输入大小的函数

```
1 def find_max(data):
2     """Return the maximum element from a nonempty Python list."""
3     biggest = data[0]           # The initial value to beat
4     for val in data:           # For each value:
5         if val > biggest       # if it is greater than the best so far,
6             biggest = val      # we have found a new best (so far)
7     return biggest             # When loop ends, biggest is the max
```

- 原子计算次数：
 - 第1行：2次；第3行：2次；第4行：2n次；第5行：2n次；第6行：0-n次；第7行：1次

估计程序运行时间



- **find_max**算法在最坏情况下运行 $5n+5$ 次原子操作，在最好情况下运行 $4n+5$ 次原子操作。如果我们定义：

a = 最快的原子运算时间

b = 最慢的原子运算时间

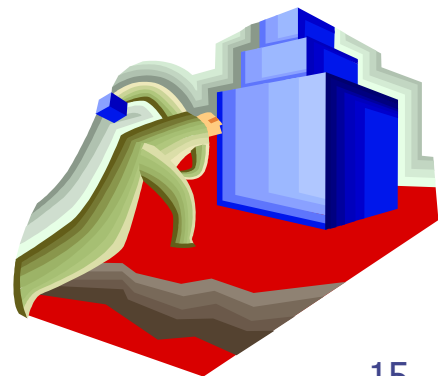
- 令 $T(n)$ 为**find_max**的运行时间，则：

$$a(4n + 5) \leq T(n) \leq b(5n + 5)$$

- 因此，程序的运行时间 $T(n)$ 被两个线性函数所约束

程序运行时间的增长率

- $T(n)$ 会如何变化?
- 如只改变软件或硬件环境, 则:
 - 只会给 $T(n)$ 带来常数因子的影响, 但……
 - 这不会改变 $T(n)$ 的增长速度
- 运行时间的线性增长特性是算法find_max的固有特性

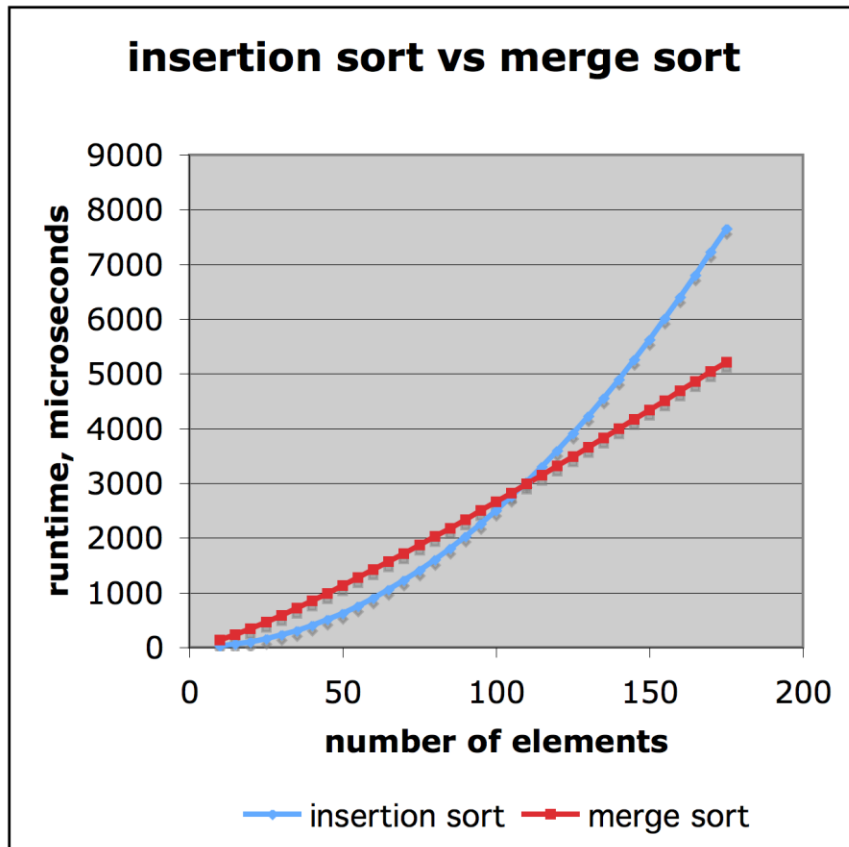


增长率的重要性

当前运行时间	问题规模为 $n+1$ 时运行时间	问题规模为 $2n$ 时运行时间	问题规模为 $4n$ 时运行时间
$c \log n$	$c \log (n + 1)$	$c (\log n + 1)$	$c(\log n + 2)$
$c n$	$c (n + 1)$	$2c n$	$4c n$
$c n \log n$	$\sim c n \log n + c n$	$2c n \log n + 2c n$	$4c n \log n + 4c n$
$c n^2$	$\sim c n^2 + 2c n$	$4c n^2$	$16c n^2$
$c n^3$	$\sim c n^3 + 3c n^2$	$8c n^3$	$64c n^3$
$c 2^n$	$c 2^{n+1}$	$c 2^{2n}$	$c 2^{4n}$

问题规模翻倍时，运行时间变为原来的4倍

例：对比两种排序算法



插入排序的操作次数：

$$n^2/4$$

归并排序的操作次数：

$$2 n \log n$$

对长为1,000,000的数组排序：

插入排序需要

大约70小时

然而……

归并排序只需要

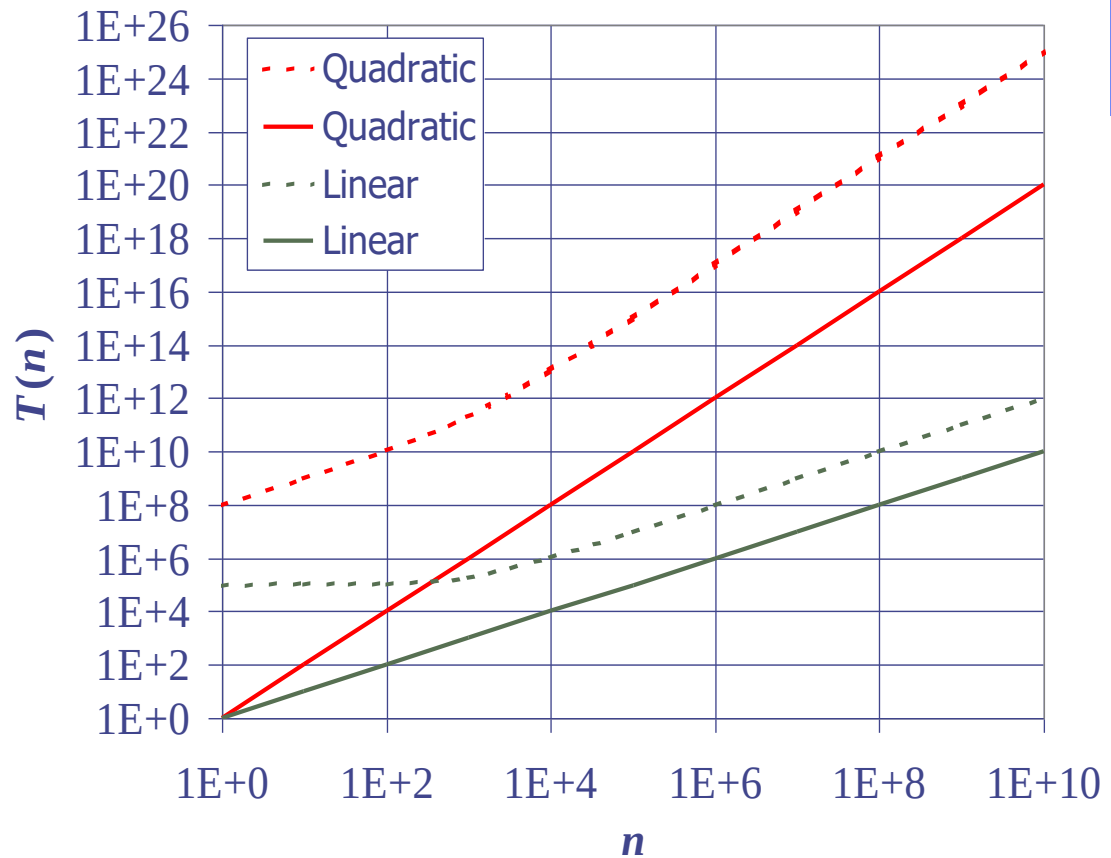
大约40秒

诚然，此计算机速度太慢，但如果其速度提高100倍，则结果会变成……

40 分钟 v.s. 少于0.5秒

常数因子及低阶项

- 运行时间增长率不受以下因素影响：
 - 常数因子
 - 低阶项
- 例：
 - $10^2n + 10^5$ 是一个线性函数
 - $10^5n^2 + 10^8n$ 是一个二次函数



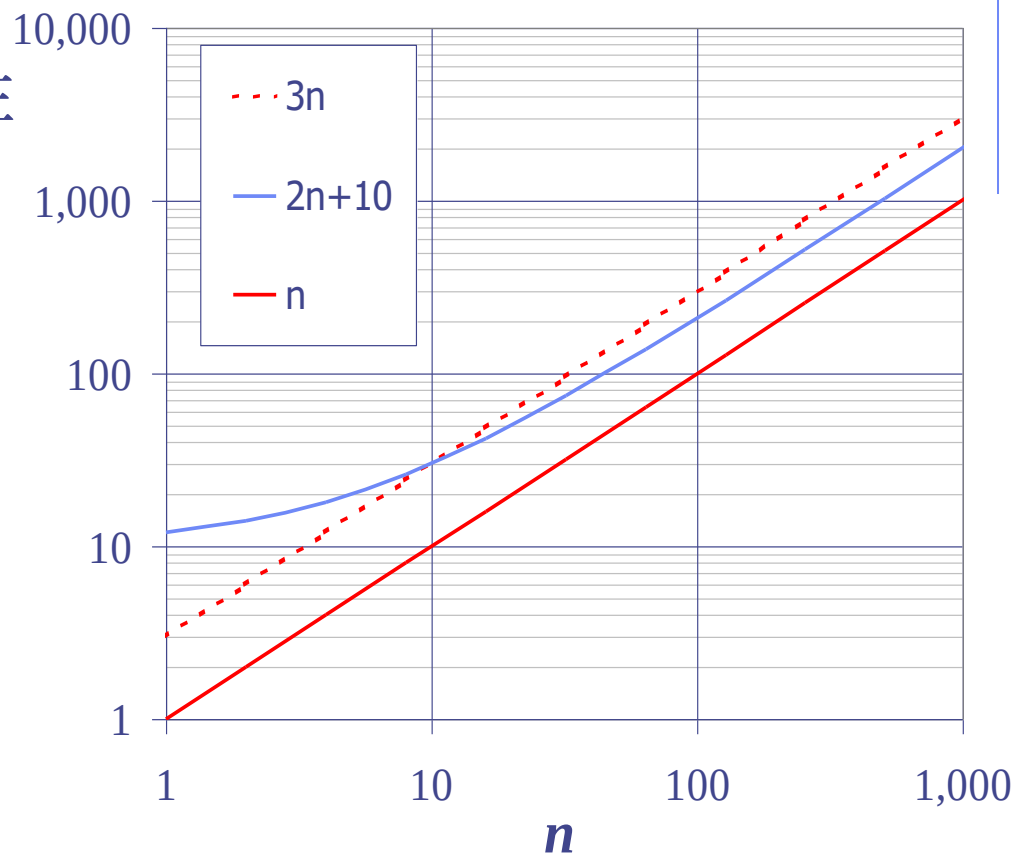
大O符号

- 对于函数 $f(n)$ 和 $g(n)$ ，如有正实数 c 和正整数 n_0 ，满足：

$$f(n) \leq cg(n), \text{ 当 } n \geq n_0$$

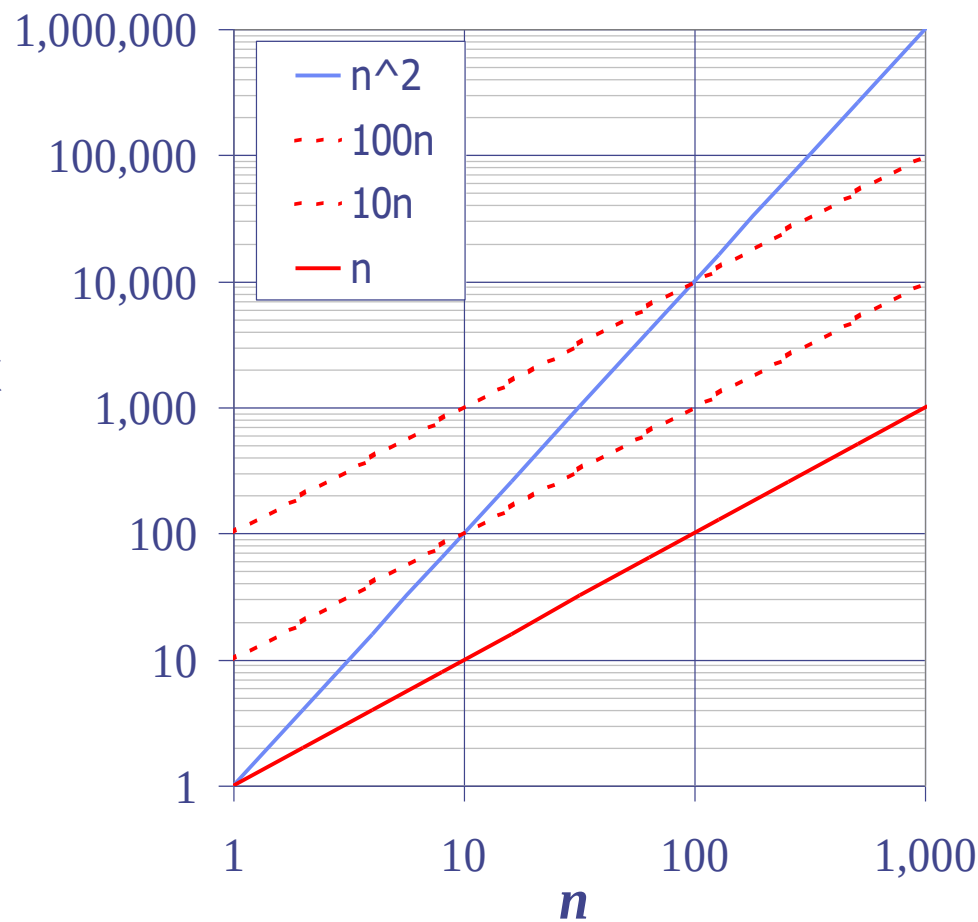
我们就说 $f(n)$ 是 $O(g(n))$

- 例： $2n+10$ 是 $O(n)$ ：
 - $2n + 10 \leq cn$
 - $(c - 2)n \geq 10$
 - $n \geq 10/(c - 2)$
 - 令 $c = 3, n_0 = 10$



大O符号

- 例：函数 n^2 不是 $O(n)$
 - $n^2 \leq cn$
 - $n \leq c$
 - 由于 c 为常数，上式无法在 n 很大时满足



大O符号

- 更多例子:
 - $7n - 2$
 - $3n^3 + 20n^2 + 5$
 - $3 \log n + 5$
 - $2n^3 + 3n \log n + 4$
 - 3^{n+1}
 -

大O与增长率

- 大O符号给出了增长率的上界
- $f(n)$ 是 $O(g(n))$ 意味着 $f(n)$ 的增长速度不会快于 $g(n)$
- 我们可以使用大O符号来对函数根据其增长率进行排序

	$f(n)$ 是 $O(g(n))$	$g(n)$ 是 $O(f(n))$
$g(n)$ 增长更快	是	否
$f(n)$ 增长更快	否	是
增长率相同	是	是

大O符号的规律与准则

- 如果 $f(n)$ 是一个指数为 d 的多项式，则 $f(n)$ 是 $O(n^d)$ ，即：
 - 忽略低阶项
 - 忽略常数因子
- 使用所有 $g(n)$ 中增长率最慢的那个
 - 如：使用“ $2n$ 是 $O(n)$ ”，而非“ $2n$ 是 $O(n^2)$ ”
- 使用所有 $g(n)$ 中最简洁的那个
 - 如：使用“ $3n + 5$ 是 $O(n)$ ”，而非“ $3n + 5$ 是 $O(3n)$ ”

算法渐进分析

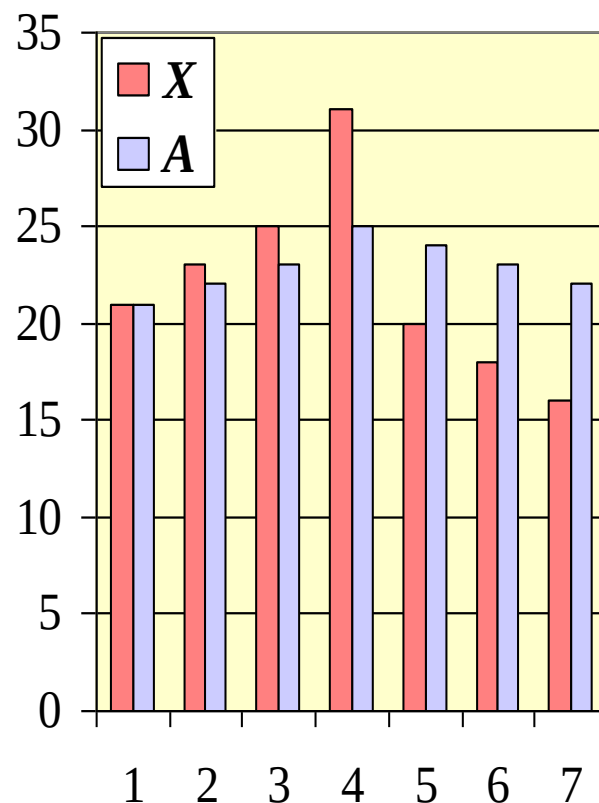
- 算法渐进分析使用大O符号来描述运行时间
- 为了进行渐进分析，我们需要：
 1. 找到最坏情况下原子操作的次数与输入大小间的函数关系
 2. 使用大O符号对此函数进行表示
- 例：
 - 算法find_max的运行时间为 $O(n)$ ”
- 低阶项和常数因子最终会被丢弃，故计算原子操作次数时可以直接忽略

计算前缀平均值

- 我们使用两种计算前缀平均值的算法来对渐进分析进行进一步的说明
- 数组 X 的第 i 个前缀平均值是其前 $(i+1)$ 个元素的平均值:

$$A[i] = (X[0] + X[1] + \dots + X[i]) / (i+1)$$

- 计算数组 X 的前缀平均值在金融分析中有应用



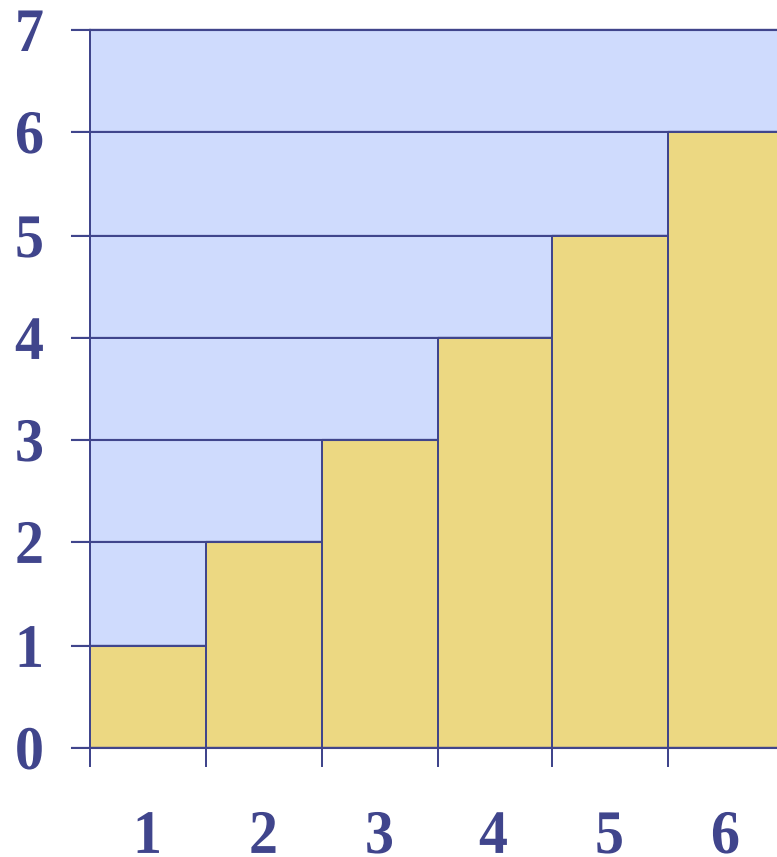
前缀平均值（二次算法）

- 下面的算法在二次时间内计算前缀平均值：

```
1 def prefix_average1(S):
2     """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
3     n = len(S)
4     A = [0] * n                                # create new list of n zeros
5     for j in range(n):
6         total = 0                                # begin computing S[0] + ... + S[j]
7         for i in range(j + 1):
8             total += S[i]
9         A[j] = total / (j+1)                    # record the average
10    return A
```

前缀平均值（二次算法）

- *prefix_average1*的运行时间为 $O(1 + 2 + \dots + n)$
- 前 n 个正整数的求和为 $n(n + 1) / 2$
 - 右图为一个直观的证明
 - 可用等差数列求和公式
- 因此，*prefix_average1*算法的运行时间为 $O(n^2)$



前缀平均值（第2种算法）

- 下面的算法采用Python内置函数来简化代码：

```
1 def prefix_average2(S):
2     """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
3     n = len(S)
4     A = [0] * n                # create new list of n zeros
5     for j in range(n):
6         A[j] = sum(S[0:j+1]) / (j+1)  # record the average
7     return A
```

前缀平均值（线性算法）

- 下面的算法通过保存一个实时的求和值，在线性时间内计算前缀平均值：

```
1 def prefix_average3(S):
2     """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
3     n = len(S)
4     A = [0] * n                # create new list of n zeros
5     total = 0                  # compute prefix sum as S[0] + S[1] + ...
6     for j in range(n):
7         total += S[j]          # update prefix sum to include S[j]
8         A[j] = total / (j+1)   # compute average based on current sum
9     return A
```

相关数学知识回顾



- 求和公式

- $$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

- $$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

- $$\sum_{i=1}^n i^k \text{ 为 } O(n^{k+1})$$

- 对数和指数:

- 对数函数的运算性质:**

$$\log_b(xy) = \log_b x + \log_b y$$

$$\log_b(x/y) = \log_b x - \log_b y$$

$$\log_b x^a = a \log_b x$$

$$\log_b a = \log_x a / \log_x b$$

- 指数函数的运算性质:**

$$a^{(b+c)} = a^b a^c$$

$$a^{bc} = (a^b)^c$$

$$a^b / a^c = a^{(b-c)}$$

$$b = a^{\log_a b}$$

$$b^c = a^{c \cdot \log_a b}$$

大O的“亲戚”们



- 大 Ω 符号

- 如果存在正实数 c 和整型常数 $n_0 \geq 1$ 满足

$$f(n) \geq cg(n), \text{ 当 } n \geq n_0$$

我们就说 $f(n)$ 是 $\Omega(g(n))$

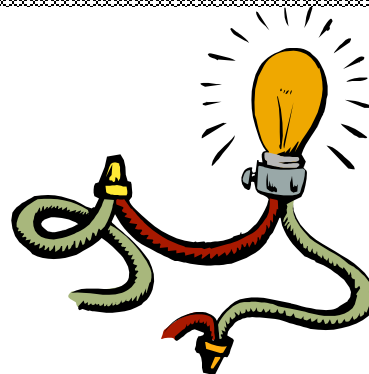
- 大 Θ 符号

- 如果存在正实数 c' 、 c'' 和一个整型常数 $n_0 \geq 1$ 满足

$$c'g(n) \leq f(n) \leq c''g(n), \text{ 当 } n \geq n_0$$

我们就说 $f(n)$ 是 $\Theta(g(n))$

渐进符号的直觉理解



大O

- 如果 $f(n)$ 渐进地小于或等于 $g(n)$ 时，我们就说 $f(n)$ 是 $O(g(n))$

大 Ω

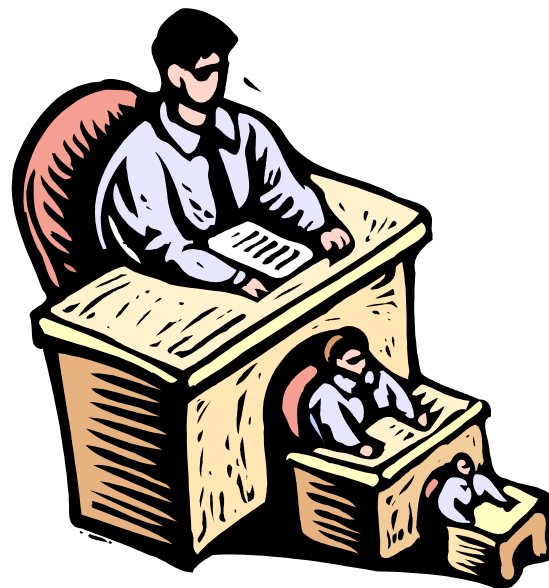
- 如果 $f(n)$ 渐进地大于或等于 $g(n)$ 时，我们就说 $f(n)$ 是 $\Omega(g(n))$

大 Θ

- 如果 $f(n)$ 渐进地等于 $g(n)$ 时，我们就说 $f(n)$ 是 $\Theta(g(n))$



递归



递归机制

- 递归: 调用其自身
- 经典例子——阶乘函数:
 - $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-1) \cdot n$
- 递归定义:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n-1) & \text{else} \end{cases}$$

- Python实现:

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else:  
5         return n * factorial(n-1)
```

递归机制

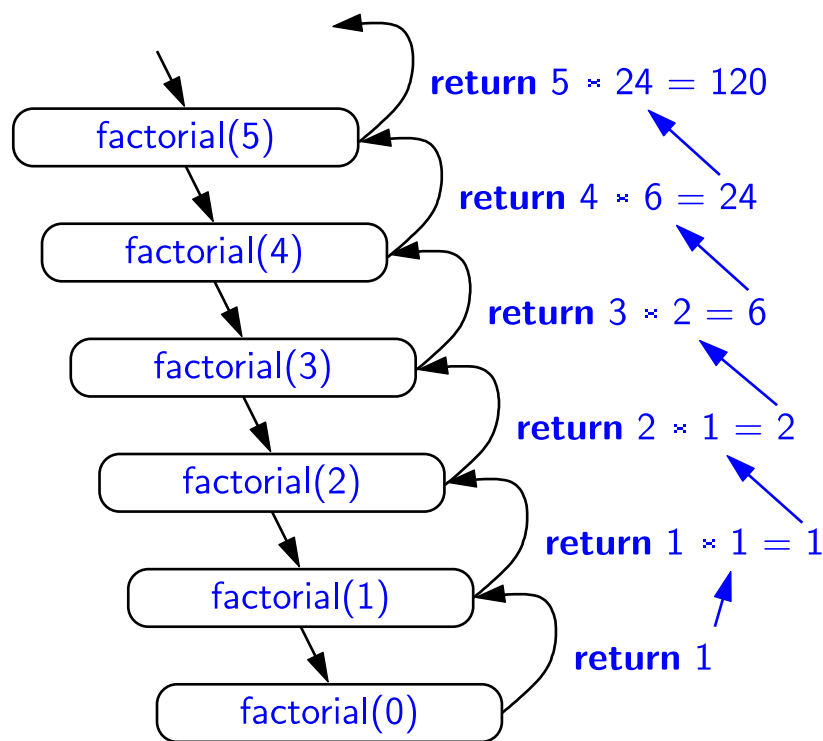
- 递归方法中通常包含两部分：**基本情况**和**递归调用**
 - **基本情况**
 - ◆ 对于不需要进行递归调用的输入值，即为基本情况（基本情况应有至少1个）
 - ◆ 每一条递归调用链都**必须**最终调用某一个基本情况
 - **递归调用**
 - ◆ 递归调用即为对当前方法自身的调用
 - ◆ 递归调用的定义方式应使得递归调用链朝某一基本情况延伸

递归的可视化

- 递归轨迹

- 每一条目代表一次递归调用
- 递归调用：箭头从递归调用者指向被调用者
- 函数值返回：箭头从递归被调用者指向调用者，箭头上显示返回值

- 例：



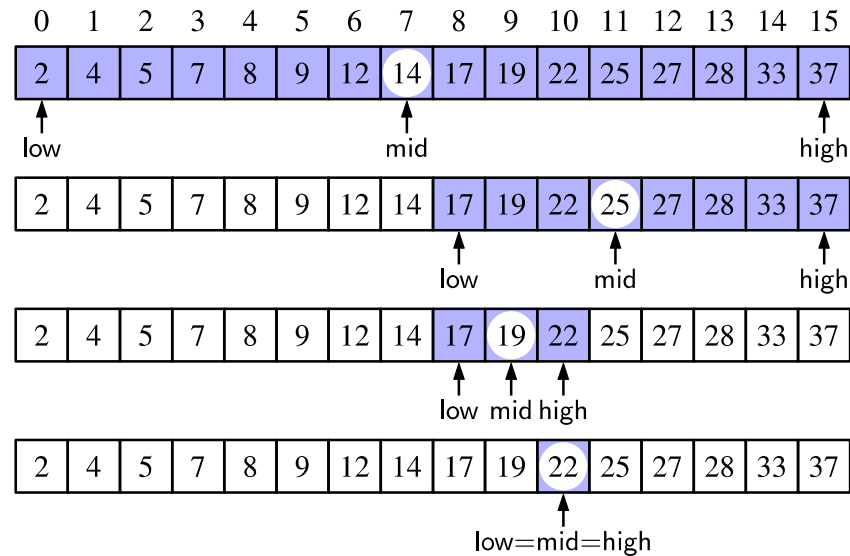
二分查找

- 二分查找是一个典型的递归算法
- 该算法用于在顺序序列中搜寻一个目标值

```
1 def binary_search(data, target, low, high):
2     """ Return True if target is found in indicated portion of a Python list.
3
4     The search only considers the portion from data[low] to data[high] inclusive.
5     """
6     if low > high:
7         return False                                # interval is empty; no match
8     else:
9         mid = (low + high) // 2
10        if target == data[mid]:                      # found a match
11            return True
12        elif target < data[mid]:
13            # recur on the portion left of the middle
14            return binary_search(data, target, low, mid - 1)
15        else:
16            # recur on the portion right of the middle
17            return binary_search(data, target, mid + 1, high)
```

二分查找的可视化

- 考虑以下三种情况：
 - 如果目标值等于 $\text{data}[\text{mid}]$ ，则我们找到了目标值
 - 如果目标值 $< \text{data}[\text{mid}]$ ，则我们在序列的前半部分重复这一查找过程
 - 如果目标值 $> \text{data}[\text{mid}]$ ，则我们在序列的后半部分重复这一查找过程



二分查找分析

- 二分查找的时间复杂度是什么？
 - 每次递归调用中，序列长度为 $high - low + 1$.
 - 一次比较后，下一个递归调用的序列长度为下面两个中的一个：

$$(mid - 1) - low + 1 = \left\lfloor \frac{low + high}{2} \right\rfloor - low \leq \frac{high - low + 1}{2}$$

$$high - (mid + 1) + 1 = high - \left\lceil \frac{low + high}{2} \right\rceil \leq \frac{high - low + 1}{2}.$$

- 因此，每次递归调用将搜索区域大小减半，最多递归调用 $\lfloor \log n \rfloor + 1$ 次，即二分查找的时间复杂度为 $O(\log n)$

递归分类

- 通过考虑在一个激活的函数体内开始的递归调用的最大数量来对递归进行分类：
 - 如果一个递归调用最多开始一个其他的递归调用，我们称之为**线性递归**
 - 如果一个递归调用可以开始两个其他的递归调用，我们称之为**二路递归**
 - 如果一个递归调用可以开始三个或以上其他的递归调用，我们称之为**多重递归**

线性递归

- 线性递归只在函数内递归调用一次
 - 递归函数内可能有多个可能的下一次递归调用的方式，但此函数内必须最多只能执行一次递归调用
 - 每一个递归调用都需要将递归链延伸到某一种基本情况
 - 例：
 - ◆ 阶乘的递归式定义
 - ◆ 二分查找
 - 注意：线性递归是指递归轨迹为线性，而非时间复杂度为线性

线性递归举例

递归求和：

Algorithm LinearSum(A, n):

Input:

An integer array A and an integer n ,
such that A has at least n elements

Output:

The sum of the first n integers in A

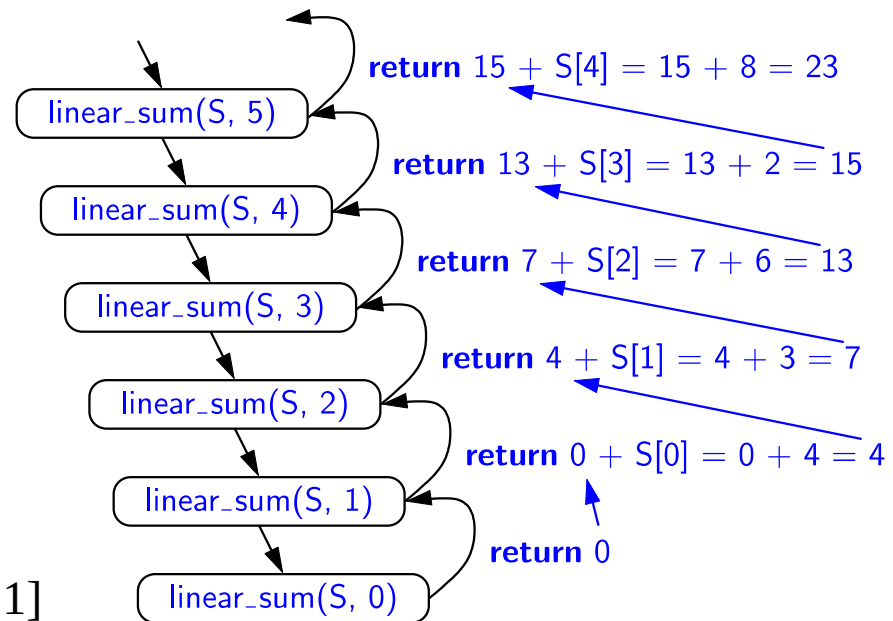
if $n = 1$ **then**

return $A[0]$

else

return LinearSum($A, n - 1$) + $A[n - 1]$

递归轨迹：



线性递归举例

- 逆置序列:

Algorithm ReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

if $i < j$ **then**

 Swap $A[i]$ and $A[j]$

 ReverseArray($A, i + 1, j - 1$)

return

0	1	2	3	4	5	6
4	3	6	2	8	9	5
5	3	6	2	8	9	4
5	9	6	2	8	3	4
5	9	8	2	6	3	4
5	9	8	2	6	3	4

为递归附加有用的参数

- 创建递归方法时，很重要的一点是，如何使得递归能更方便的进行
- 这使得我们有时需要在递归函数中**附加其他参数**
- 因此在本例中，我们使用的方法为ReverseArray(A, i, j)，而非ReverseArray(A)
- Python实现如下：

```
1 def reverse(S, start, stop):  
2     """ Reverse elements in implicit slice S[start:stop]. """  
3     if start < stop - 1:                                # if at least 2 elements:  
4         S[start], S[stop-1] = S[stop-1], S[start]      # swap first and last  
5         reverse(S, start+1, stop-1)                    # recur on rest
```

二路递归

- 当除基本情况外，每个函数执行两次递归调用时，即为二路递归
- 例：递归求和：

Algorithm BinarySum(A, i, n):

Input: An array A and integers i and n

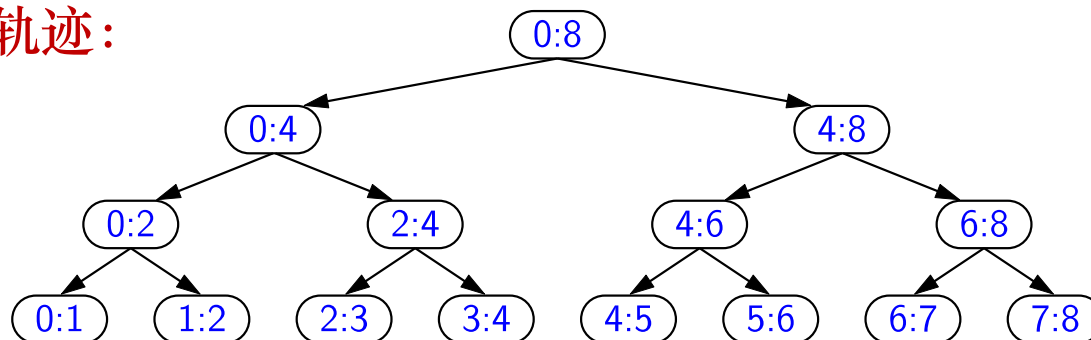
Output: The sum of the n integers in A starting at index i

if $n = 1$ **then**

return $A[i]$

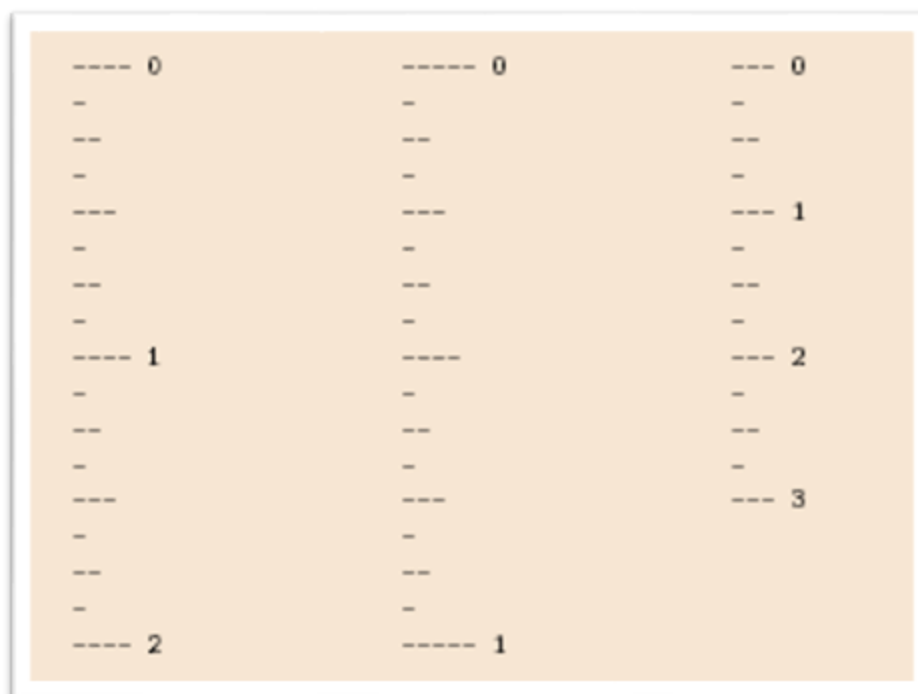
return BinarySum($A, i, n/2$) + BinarySum($A, i + n/2, n/2$)

- 递归轨迹：



二路递归举例

- 绘制英制标尺上的数字标签及刻度线：



二路递归举例

- 计算斐波那契数列，其递推式为：

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i > 1.$$

- 二路递归算法：

Algorithm BinaryFib(k)

Input: Nonnegative integer

Output: The k -th Fibonacci number F_k

if $k = 1$ **then**

return k

else

return BinaryFib($k - 1$) + BinaryFib($k - 2$)

二路递归举例

- 不幸的是，这样的递归实现效率非常低
- 令 n_k 表示BinaryFib(k)函数的递归调用次数，则：
 - $n_0 = 1$
 - $n_1 = 1$
 - $n_2 = n_1 + n_0 + 1 = 1 + 1 + 1 = 3$
 - $n_3 = n_2 + n_1 + 1 = 3 + 1 + 1 = 5$
 - $n_4 = n_3 + n_2 + 1 = 5 + 3 + 1 = 9$
 - $n_5 = n_4 + n_3 + 1 = 9 + 5 + 1 = 15$
 - $n_6 = n_5 + n_4 + 1 = 15 + 9 + 1 = 25$
 - $n_7 = n_6 + n_5 + 1 = 25 + 15 + 1 = 41$
 - $n_8 = n_7 + n_6 + 1 = 41 + 25 + 1 = 67$
- n_k 的下标每增长2，其值变为2倍以上
- 这意味着 $n_k > 2^{k/2}$ ，即此算法的时间复杂度为指数级！

斐波那契算法改进

- 增加参数数目，将二路递归变为线性递归，减少递归调用次数：

Algorithm **LinearFibonacci**(k):

Input: A nonnegative integer k

Output: Pair of Fibonacci numbers (F_k, F_{k-1})

if $k = 1$ **then**

return $(k, 0)$

else

$(i, j) = \text{LinearFibonacci}(k - 1)$

return $(i + j, i)$

- **LinearFibonacci**只产生 $k - 1$ 次递归调用

多重递归

- 多重递归中的函数可能执行多于两次递归调用
- 多重递归常见应用：求和谜题

pot + pan = bib

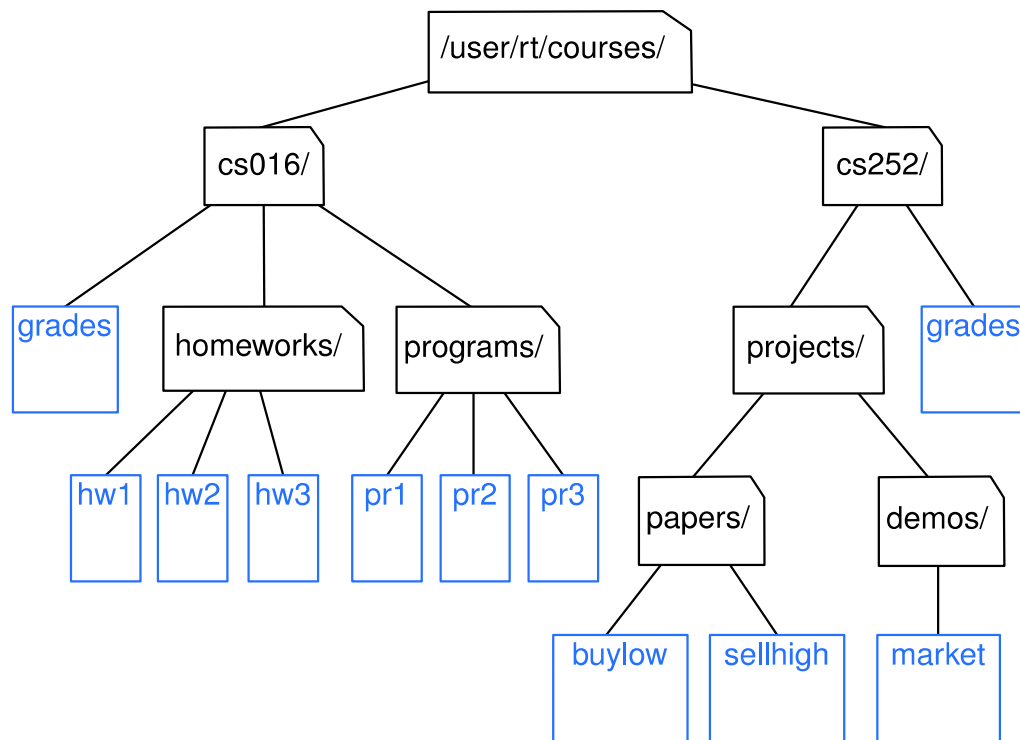
dog + cat = pig

boy + girl = baby

- 给方程中的每一个字母分配一个数字，使方程为真

多重递归举例

- 文件系统拥有递归特性
- 如何计算一个特定目录下的所有文件和目录占有的总磁盘空间？



多重递归举例

- 计算目录空间的多重递归算法:

Algorithm **DiskUsage**(path):

Input: A string designating a path to a file-system entry

Output: The cumulative disk space used by that entry and any nested entries

total = size(path) {immediate disk space used by the entry}

if path represents a directory **then**

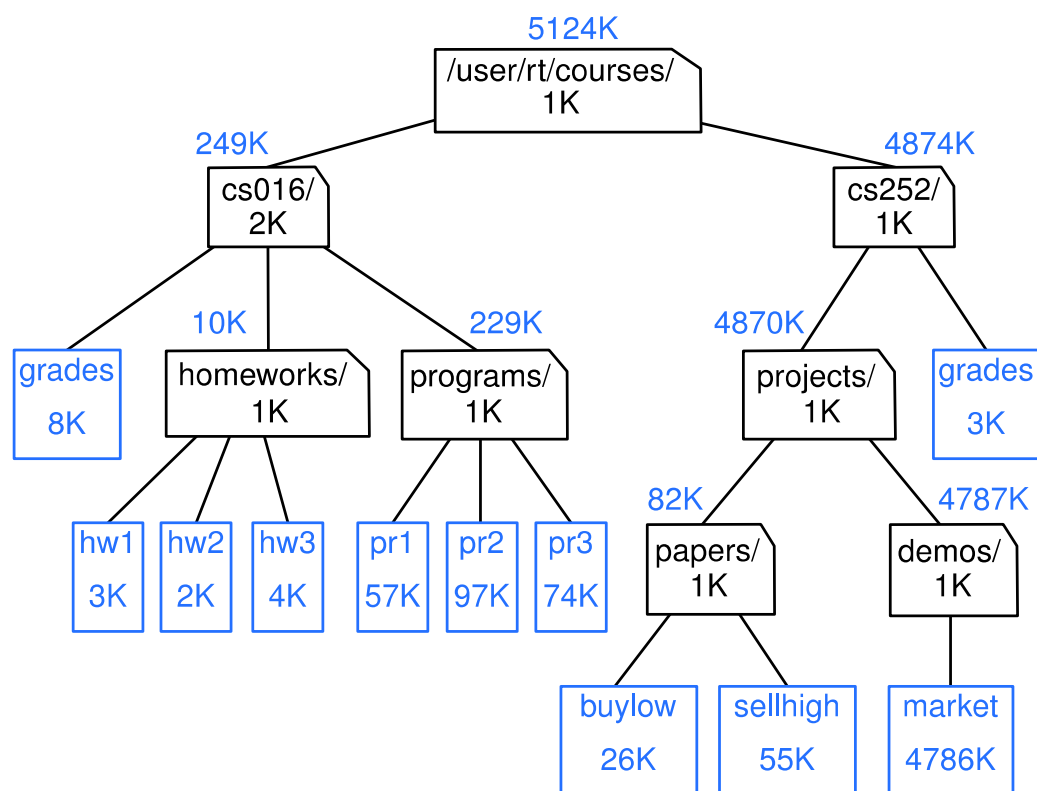
for each child entry stored within directory path **do**

 total = total + DiskUsage(child) {recursive call}

return total

多重递归举例

• 例如：



尾递归

- 递归算法的优点在于**简洁**地利用许多问题中的重复结构
- 然而，递归的使用**成本**可能不低。Python解释器必须跟踪每个嵌套调用的状态的活动记录。当计算机内存价格不菲时，这些嵌套调用造成的成本上升
- 某些情况下，从**递归算法**得到**非递归算法**是很有用的
- **尾递归**是一种较容易被转化为非递归算法的递归形式

尾递归

- 当递归算法进行的递归调用是此算法的**最后一步**，即递归调用得到的值被立即当作此算法的输出，则此递归为一个**尾递归**
- 根据定义，尾递归必须是一个**线性递归**
- 例：

- 序列逆置算法：

Algorithm IterativeReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

while $i < j$ **do**

 Swap $A[i]$ and $A[j]$

$i = i + 1$

$j = j - 1$

return

0	1	2	3	4	5	6
4	3	6	2	8	9	5
5	3	6	2	8	9	4
5	9	6	2	8	3	4
5	9	8	2	6	3	4
5	9	8	2	6	3	4

尾递归

- 思考：我们学过的哪些算法是尾递归？
 - 阶乘函数？二分查找？递归求和？斐波那契数列？
- 二分查找算法的非递归实现：

```
1 def binary_search_iterative(data, target):
2     """ Return True if target is found in the given Python list."""
3     low = 0
4     high = len(data)-1
5     while low <= high:
6         mid = (low + high) // 2
7         if target == data[mid]:           # found a match
8             return True
9         elif target < data[mid]:
10            high = mid - 1                 # only consider values left of mid
11        else:
12            low = mid + 1                  # only consider values right of mid
13    return False                          # loop ended without success
```



Enjoy